TASK 24 – DOCKER CONTAINERIZATION: INSTALLATION, CUSTOM IMAGE BUILD AND PERSISTENT WEB HOSTING ON AWS

Objective

1. Study the basics of the Docker containerization platform.
2. Install Docker on a RHEL-based AWS EC2 instance and verify the installation.
3. Build and run a container from an available Docker image.
4. Study and practise the basic Docker commands.
5. Host a website using a container on an AWS instance.
6. Create a new custom Docker image using a Dockerfile and host the website from that image.
7. Demonstrate that data is not lost when a container is removed, using Docker volumes.

1. Docker Basics

Docker is a containerization platform that packages an application and its dependencies into a portable unit called a container. Containers share the host operating system kernel, so they start in seconds and use fewer resources than virtual machines.

Key concepts:

Term	Meaning
Image	A read-only template containing the application and its dependencies, built in layers
Container	A running instance of an image
Registry	A storage location for images (Docker Hub is the default)
Dockerfile	A text file with instructions for building an image
Docker daemon	The background service that builds and runs containers
Port mapping	Connects a port on the host to a port inside the container (-p host:container)
Volume	Storage managed by Docker, kept outside the container so data survives container removal

2. Environment Preparation

A RHEL EC2 instance was launched and its security group was configured to allow web access. The system packages were updated before installing Docker.

Steps:

1. In the EC2 security group, add inbound rules for SSH (port 22) and HTTP (port 80).
2. Connect to the instance using SSH:

Bash

ssh -i your-key.pem ec2-user@<public-ip>

3. Update the system packages:

Bash

sudo dnf update -y

1. Install the DNF plugins package (provides the config-manager tool):

Bash

sudo dnf install -y dnf-plugins-core

3. Docker Repository Configuration and Installation

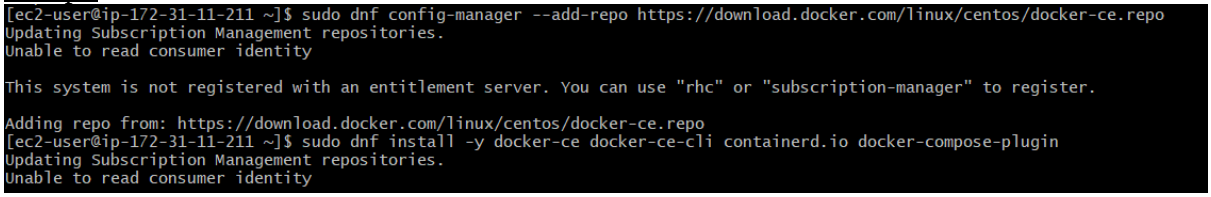
Docker is not included in the default RHEL repositories, so the official Docker repository was added before installing the packages.

Steps:

1. Add the Docker repository:

Bash

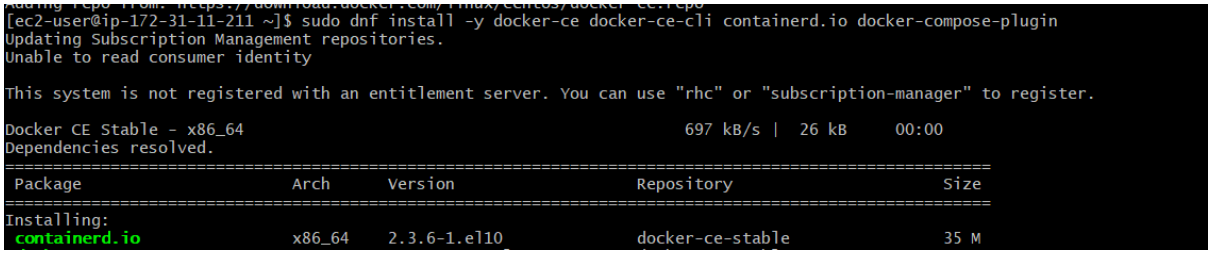
sudo dnf config-manager --add-repo https://download.docker.com/linux/centos/docker-ce.repo



2. Install Docker:

Bash

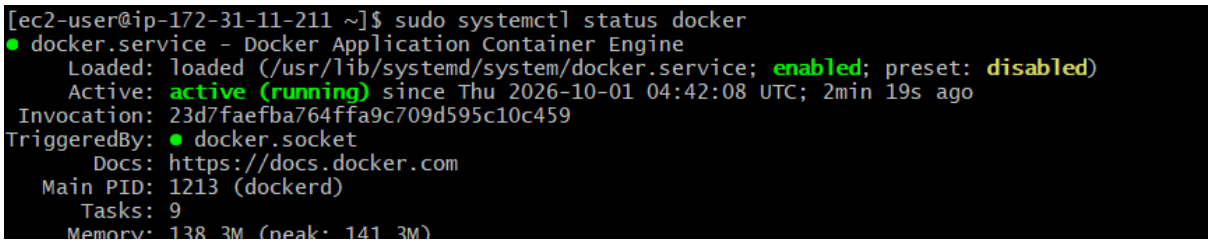
sudo dnf install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin



3. Start Docker and enable it on boot:

Bash

sudo systemctl enable --now docker
sudo systemctl status docker



4. Allow ec2-user to run Docker commands without sudo, then log out and log back in:

Bash

sudo usermod -aG docker ec2-user
exit

5. Verify the installation:

Bash

docker --version
docker run hello-world

The message **"Hello from Docker!"** confirmed that the installation was successful.

```
[ec2-user@ip-172-31-11-211 ~]$ docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:5e230903533324d887c48ad5e5c56d294eab81588df9605b07d1afe895f9cc8f8
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.
```

4. Running a Container from an Existing Image

The official Apache HTTP Server image (httpd) was downloaded from Docker Hub and run as a container.

Steps:

1. Download the image and list the images:

Bash

```
docker pull httpd
docker images
```

2. Run a container in the background with port 80 mapped:

Bash

```
docker run -d --name myapache -p 80:80 httpd
```

```
[ec2-user@ip-172-31-11-211 ~]$ docker pull httpd
Using default tag: latest
latest: Pulling from library/httpd
7a3bdd751d95: Pull complete
1320c4426edf: Pull complete
4f4fb700ef54: Pull complete
6b37362b3da7: Pull complete
6db0cf540580: Pull complete
2f4636dca226: Pull complete
fdff2e4882b0: Download complete
0b4153e35dab: Download complete
Digest: sha256:03f858efb82c25cb0f9962946615cdcdaf26927cb971722149b563197cfd0fdd
Status: Downloaded newer image for httpd:latest
docker.io/library/httpd:latest
[ec2-user@ip-172-31-11-211 ~]$ docker run -d --name myapache -p 80:80 httpd
6d5c2d3e0cc8772d7e65c10be86e17e776b803a39900225a80d5d685e39e746b
[ec2-user@ip-172-31-11-211 ~]$ docker ps
CONTAINER ID   IMAGE     COMMAND              CREATED          STATUS          PORTS                                   NAMES
6d5c2d3e0cc8   httpd     "httpd-foreground"   30 seconds ago   Up 30 seconds   0.0.0.0:80->80/tcp, [::]:80->80/tcp   myapache
```

3. Verify that the container is running:

Bash

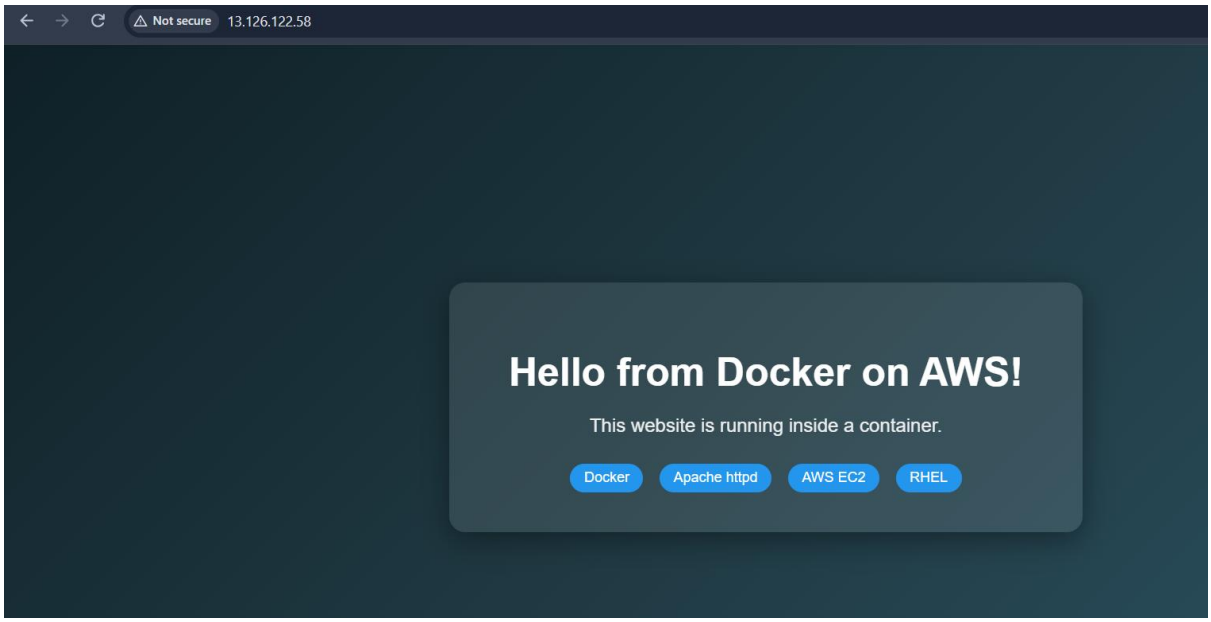
```
docker ps
```

4. Test the website from the server and then from a browser at http://<ec2-public-ip>:

Bash

curl http://localhost

The default Apache page ("It works!") was displayed.



6. Creating a Custom Docker Image

To host the website from a new image instead of editing a running container, a custom image was built using a Dockerfile.

Steps:

1. Create a project folder and move into it:

Bash

mkdir -p ~/mysite
cd ~/mysite

2. Copy the web page out of the existing container:

Bash

docker cp myapache:/usr/local/apache2/htdocs/index.html .

```
[ec2-user@ip-172-31-11-211 mysite]$ docker cp myapache:/usr/local/apache2/htdocs/index.html .
Successfully copied 1.16kB (transferred 3.07kB) to /home/ec2-user/mysite/.
[ec2-user@ip-172-31-11-211 mysite]$ ls
index.html
[ec2-user@ip-172-31-11-211 mysite]$ head -5 index.html
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
```

3. Create the Dockerfile:

Bash

cat > Dockerfile << 'EOF'
FROM httpd:latest
COPY index.html /usr/local/apache2/htdocs/index.html
EXPOSE 80
EOF

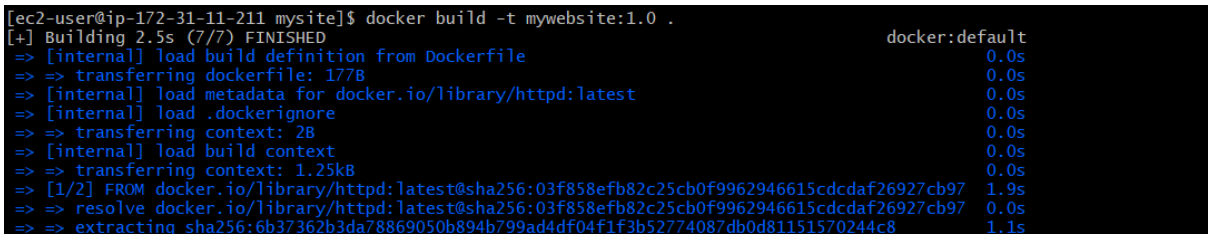
Dockerfile line	Meaning
FROM httpd:latest	Use the official Apache image as the base
COPY index.html ...	Add the website file to Apache's document root
EXPOSE 80	Document that the container listens on port 80

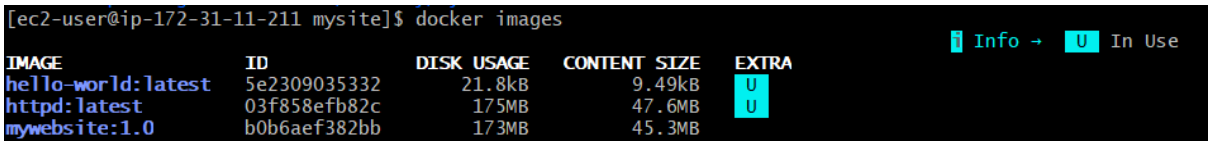
```
[ec2-user@ip-172-31-11-211 mysite]$ cat > Dockerfile << 'EOF'
FROM httpd:latest
COPY index.html /usr/local/apache2/htdocs/index.html
EXPOSE 80
EOF
[ec2-user@ip-172-31-11-211 mysite]$ cat Dockerfile
FROM httpd:latest
COPY index.html /usr/local/apache2/htdocs/index.html
EXPOSE 80
```

4. Build the image and confirm it exists:

Bash

docker build -t mywebsite:1.0 .
docker images





5. Stop the old container to free port 80, then run a container from the new image:

Bash

docker stop myapache
docker run -d --name mywebsite-container -p 80:80 mywebsite:1.0
docker ps

The IMAGE column showed mywebsite:1.0, confirming that the website was hosted from the custom image.

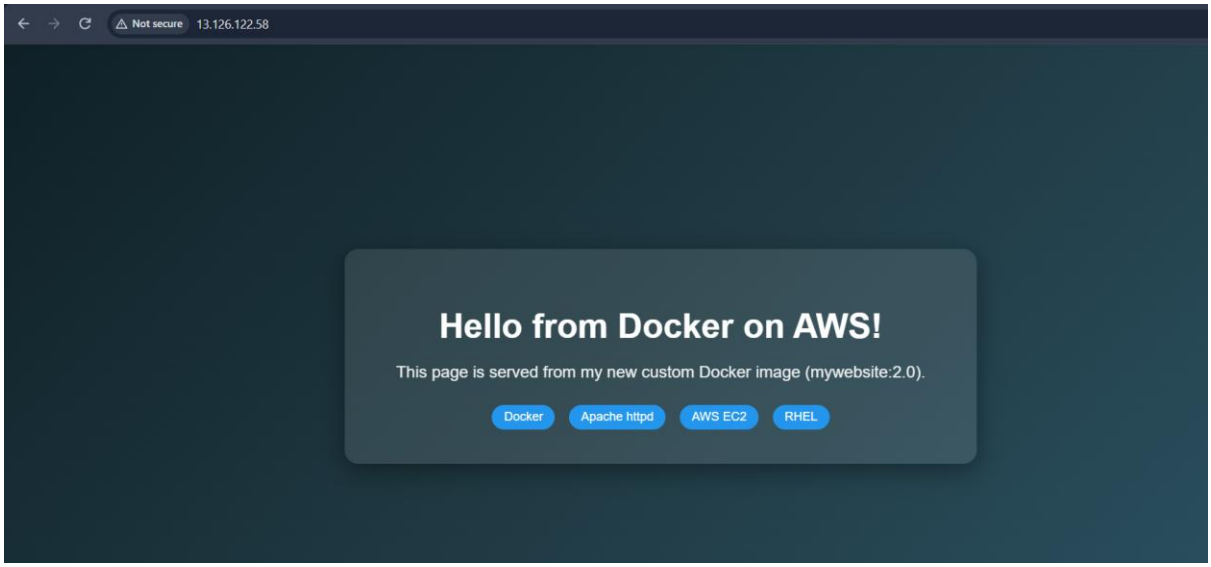


6. To confirm that content changes go through the image, the page text was edited and a new version was built:

Bash

sed -i 's|This website is running inside a container.|This page is served from my new custom Docker image (mywebsite:2.0).|' index.html
docker build -t mywebsite:2.0 .
docker stop mywebsite-container
docker run -d --name mywebsite-v2 -p 80:80 mywebsite:2.0

The browser showed the new message, confirming the page was served from mywebsite:2.0.



7. Data Persistence Using Docker Volumes

Requirement: data must not be lost when a container is removed.

Observation: files created inside a container are stored in that container's own writable layer. They survive docker stop and docker start, but they are deleted along with the container when docker rm is used. To keep data, a Docker volume is used. It is stored on the server outside any container.

Steps:

1. Create a volume:

Bash

docker volume create mydata
docker volume ls

```
[ec2-user@ip-172-31-11-211 ~]$ cd ~/mysite
[ec2-user@ip-172-31-11-211 mysite]$ docker volume create mydata
mydata
[ec2-user@ip-172-31-11-211 mysite]$ docker volume ls
DRIVER    VOLUME NAME
local     mydata
```

2. Run a container from the custom image with the volume attached at /data:

Bash

docker run -d --name voltest -v mydata:/data mywebsite:2.0

```
[ec2-user@ip-172-31-11-211 mysite]$ docker run -d --name voltest -v mydata:/data mywebsite:2.0
6f6d07af5c1afa15175fceec73ddf34f3cd04fc09ec1a77197d02962cb37e344
[ec2-user@ip-172-31-11-211 mysite]$ docker ps
```

3. Create a file inside the volume folder:

Bash

docker exec -it voltest bash
echo "This data is stored in a volume" > /data/test.txt
cat /data/test.txt
exit

```
[ec2-user@ip-172-31-11-211 mysite]$ docker exec -it voltest bash
root@6f6d07af5c1a:/usr/local/apache2# echo "This data is stored in a volume" > /data/test.txt
root@6f6d07af5c1a:/usr/local/apache2# cat /data/test.txt
This data is stored in a volume
root@6f6d07af5c1a:/usr/local/apache2# exit
exit
```

4. Delete the container and confirm the volume still exists:

Bash

docker rm -f voltest
docker ps -a
docker volume ls

5. Start a new container with the same volume and read the file:

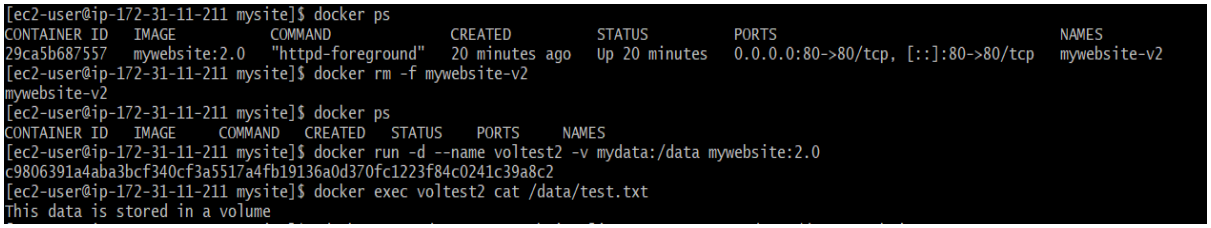
Bash

docker run -d --name voltest2 -v mydata:/data mywebsite:2.0
docker exec voltest2 cat /data/test.txt

Output:

This data is stored in a volume

The file created by the deleted container was still available in the new container, so no data was lost.



Conclusion

This task successfully demonstrated the complete lifecycle of working with Docker on AWS: installing Docker from the official repository, running containers from an existing image, practising the essential Docker commands, building a custom image with a Dockerfile, hosting a website from that image, and making data persistent with volumes so that it survives container deletion.